

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №3
по дисциплине
«Алгоритмы и структуры данных»

Выполнил студент гр. ИВТб-1303-06-00	_____ /Гортоломей И.К./
Проверил заведующий кафедрой ЭВМ	_____ /Долженкова М.Л./

Киров
2026

Цель работы:

Формирование навыков алгоритмического мышления посредством решения набора задач с predetermined характеристиками и уровнем сложности

Ссылка на профиль пользователя:

<https://codeforces.com/profile/GIKExe>

Решение заданий

Задание 1869B:

Цель задания – найти минимальную общую стоимость билетов из города А в город В. Если оба города крупные – то между ними перелёт бесплатный. Иначе цена перелёта равна расстоянию между точками. На вход подаётся список точек, первые К точек - крупные города.

Ссылка на задание:

<https://codeforces.com/contest/1869/problem/B>

Это задание решается путём поиска ближайшего к А - Х крупного города и ближайшего к В - Y крупного города. То есть, является ли экономически выгодным лететь в ближайший крупный город чтобы через него добраться в ближайший крупный город к точке назначения. Если путь до ближайшего крупного города (от А до Х + от Y до В) больше чем путь от А до В то проще сразу напрямую лететь.

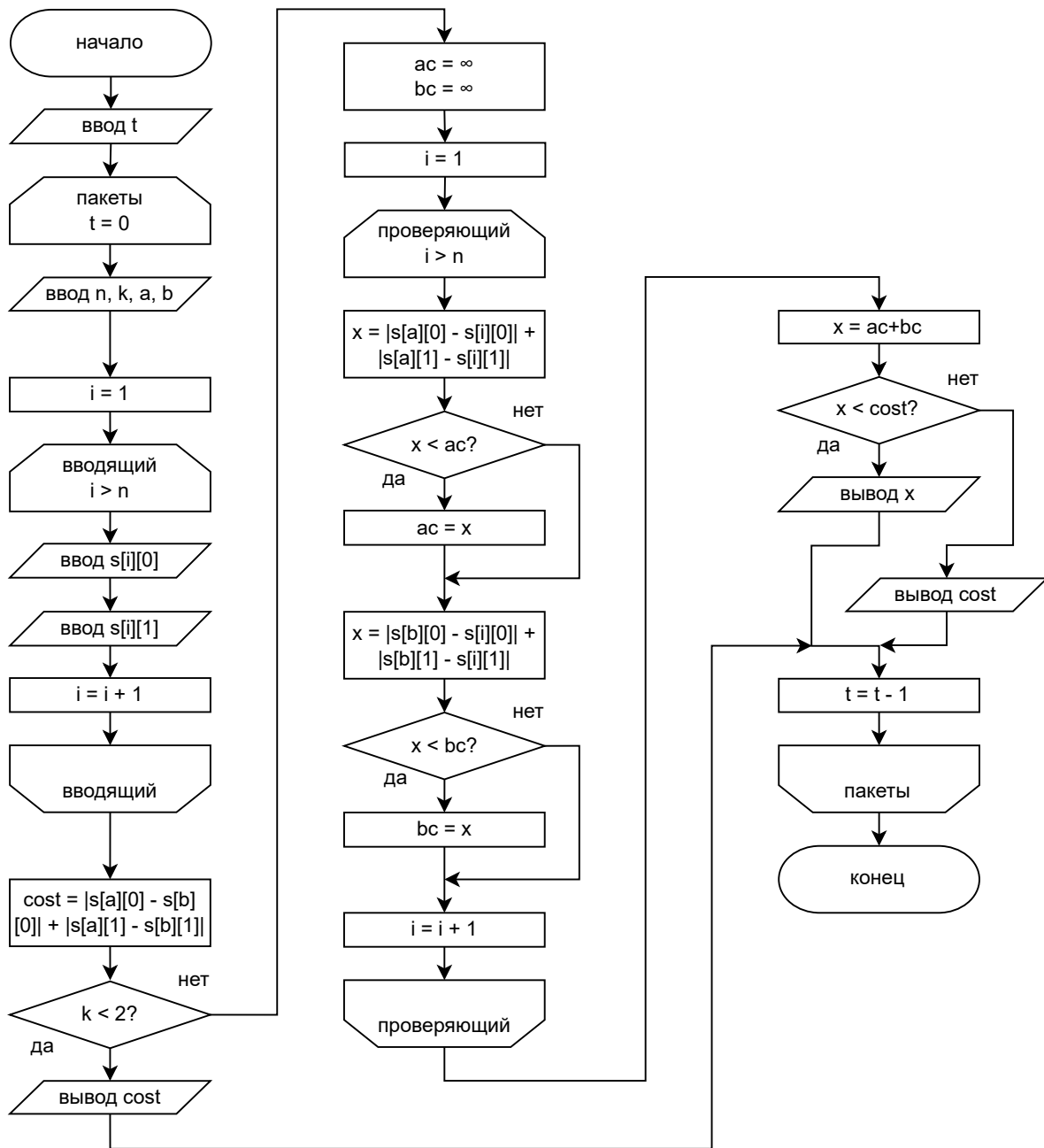


Рис. 1: Схема алгоритма задачи

Листинг 1: Исходный код программы (C)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdbool.h>
4
5  typedef unsigned long long u64;
6  #define LLONG_MAX 0x7fffffffffffffffLL
7  #define get_cost(a, b) (u64)abs(s[a][0] - s[b][0]) + (u64)abs(s[a][1] - s[b][1])
8
9  int main() {
10     int t;
11     scanf("%d", &t);
12     for (; t > 0; t--) {
13         int n, k, a, b;
14         scanf("%d %d %d %d", &n, &k, &a, &b);
15         int s[n+1][2];
16         s[0][0] = 0;
17         s[0][1] = 0;
18         for (int i = 1; i <= n; i++)
19             scanf("%d %d", &s[i][0], &s[i][1]);
20         u64 _cost, cost = get_cost(a, b);
21         if (k < 2) {
22             printf("%lld\n", cost);
23             continue;
24         }
25
26         u64 ac = LLONG_MAX, bc = LLONG_MAX;
27         for (int i = 1; i <= k; i++) {
28             _cost = get_cost(a, i);
29             if (_cost < ac) ac = _cost;
30             _cost = get_cost(b, i);
31             if (_cost < bc) bc = _cost;
32         }
33
34         _cost = ac+bc;
```

```

35     if (_cost < cost) {
36         printf("%lld\n", _cost);
37     } else {
38         printf("%lld\n", cost);
39     }
40 }
41 return 0;
42 }

```

Ссылка на успешное отправление:

<https://codeforces.com/contest/1869/submission/371533360>

Задание 520B:

Цель задания – узнать сколько раз нужно нажать кнопки чтобы прийти из числа N в число M имея операции: «умножение на 2» и «вычитание 1».

Ссылка на задание:

<https://codeforces.com/contest/520/problem/B>

Для данного задания оптимальный алгоритм:

Пока $M > N$:

Если M чётное: поделить M на 2

Если M нечётное: вычесть 1 из M

Счётчик увеличить на 1.

Когда $M \leq N$:

Вывести разницу $(N - M) + \text{счётчик}$.

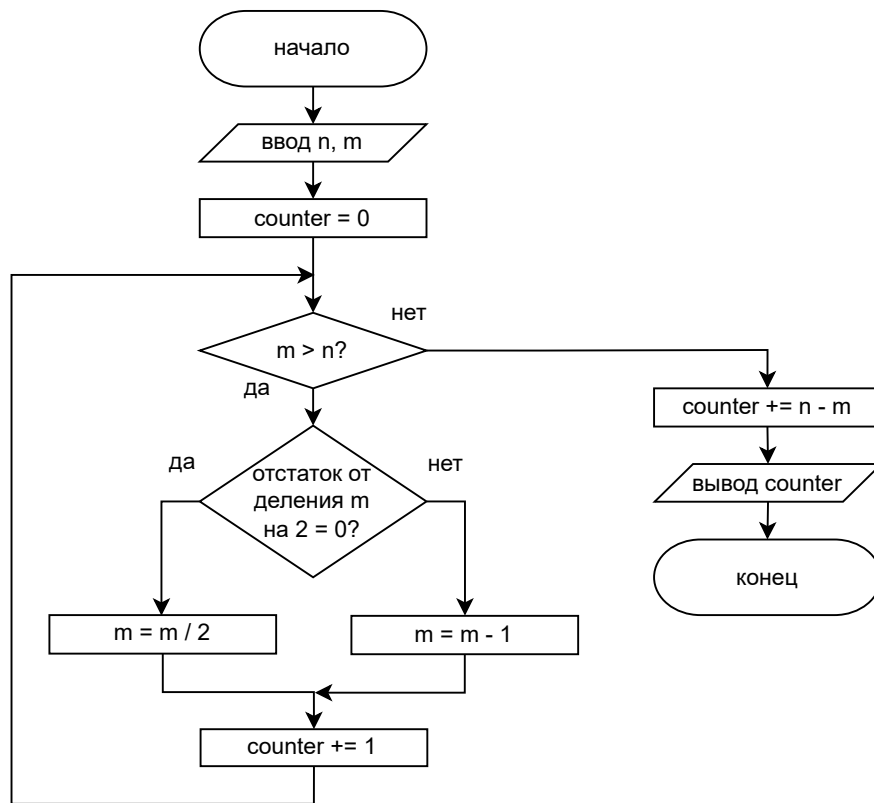


Рис. 2: Схема алгоритма задачи

Листинг 2: Исходный код программы (C)

```

1 // https://codeforces.com/contest/520/problem/B
2 #include <stdio.h>
3
4 int main() {
5     int n, m, counter = 0;
6     scanf("%d %d", &n, &m);
7     while (m > n) {
8         if (m % 2 == 0) {
9             m /= 2;
10        } else {
11            m++;
12        }
13        counter++;
14    }
  
```

```
15     counter += n - m;  
16     printf("%d", counter);  
17     return 0;  
18 }
```

Ссылка на успешное отправдение:

<https://codeforces.com/contest/520/submission/371758129>

Задание 932В:

Суть задачи: для каждого числа от 1 до 1 млн вычисляется цифровой корень $g(x)$, сводящий число к однозначному путём многократного перемножения его ненулевых цифр. В ответе нужно для каждого из Q запросов посчитать, сколько чисел на отрезке $[l, r]$ имеют заданное значение $g(x) = k$, $(1 \leq k \leq 9)$.

Ссылка на задание:

<https://codeforces.com/contest/932/problem/B>

Для решения задачи нужно предварительно вычислить для каждого числа от 1 до максимальной границы его однозначный корень, получаемый многократным перемножением ненулевых цифр. Затем для каждого из девяти возможных корней строится массив префиксных сумм, где элемент с индексом i хранит количество чисел от 1 до i с данным корнем. После этого каждый запрос на отрезке обрабатывается за $O(1)$ вычитанием префиксных значений: результат для границы R минус результат для $L-1$.

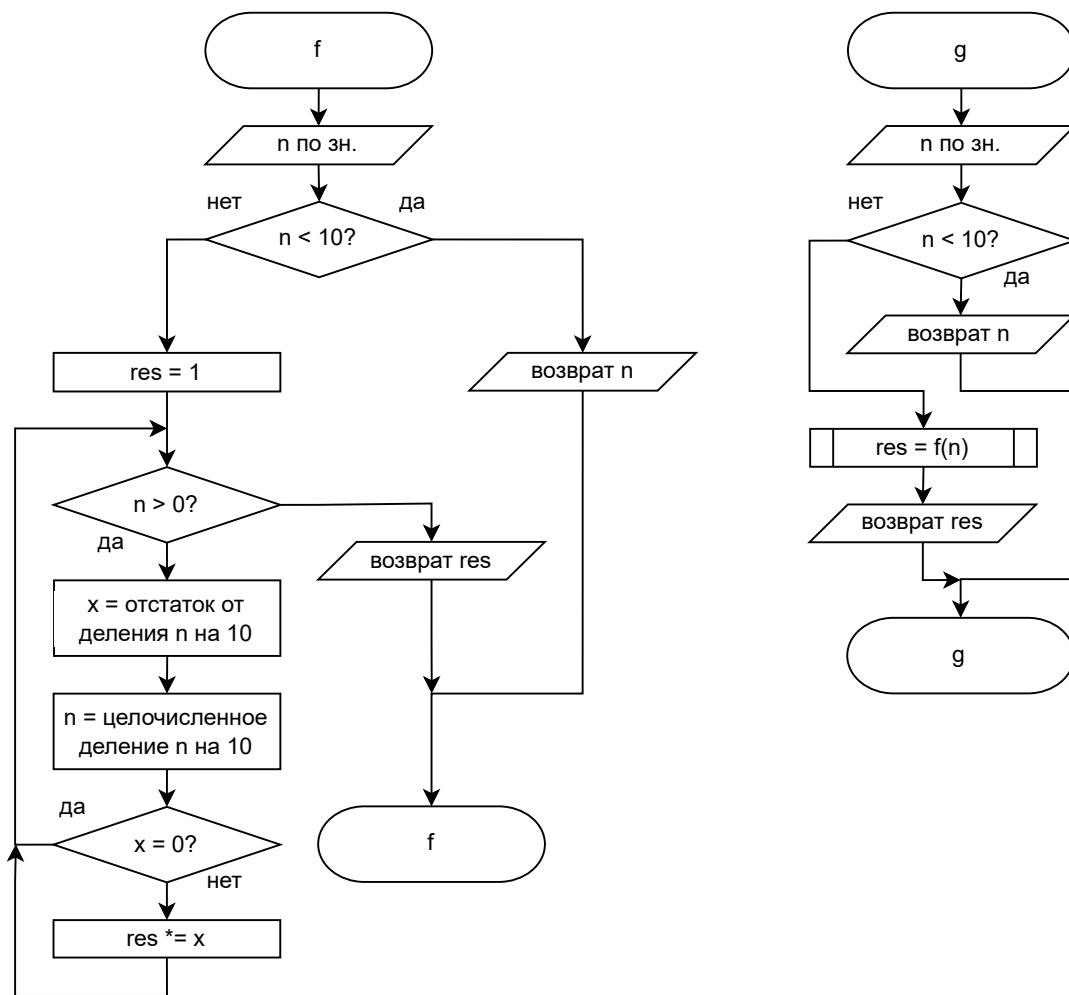


Рис. 3: Схема алгоритма задачи ч.1

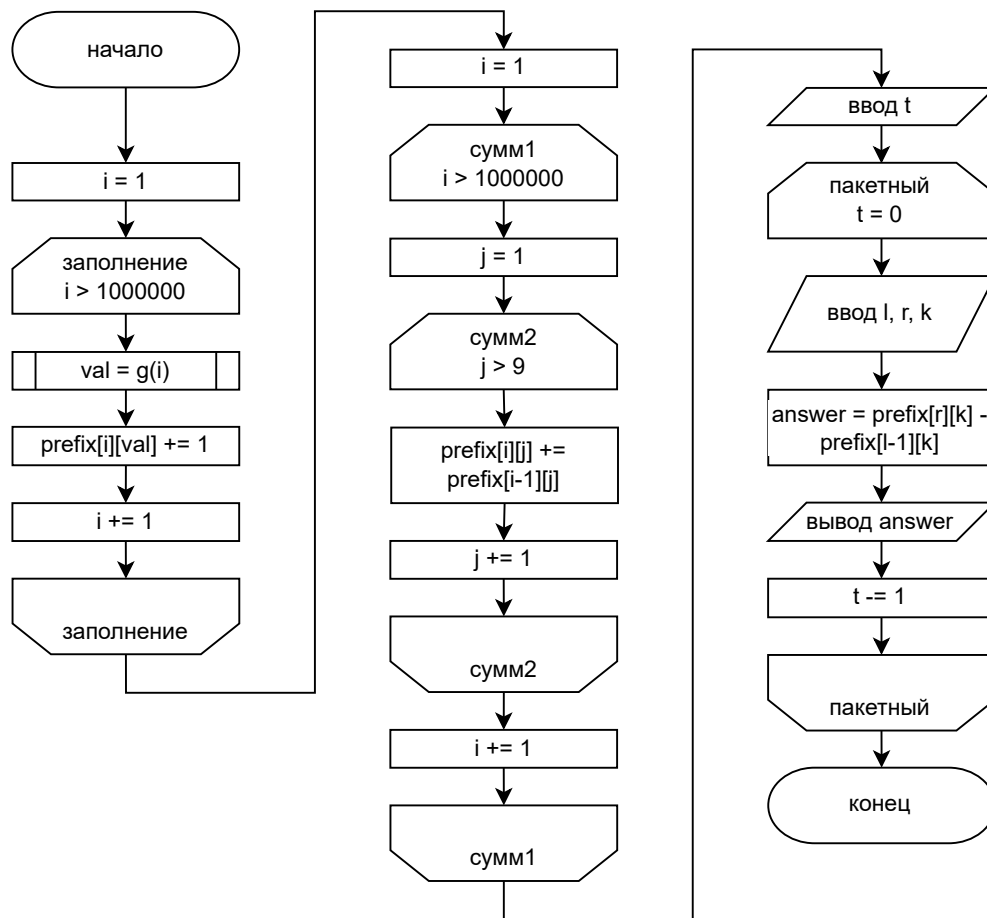


Рис. 4: Схема алгоритма задачи ч.2

Листинг 3: Исходный код программы (C)

```
1  #include <stdio.h>
2
3  #define MAX 1000000
4
5  int f(int n) {
6      if (n < 10) return n;
7      int res = 1;
8      while (n > 0) {
9          int x = n % 10;
10         n /= 10;
11         if (x == 0) continue;
12         res *= x;
13     }
14     return res;
15 }
16
17 int g(int n) {
18     if (n < 10) return n;
19     return g(f(n));
20 }
21
22 int prefix[MAX+1][10];
23
24 int main() {
25     for (int i = 1; i <= MAX; ++i) {
26         int val = g(i);
27         prefix[i][val]++;
28     }
29
30     for (int i = 1; i <= MAX; ++i) {
31         for (int j = 1; j <= 9; ++j)
32             prefix[i][j] += prefix[i-1][j];
33     }
34
35     int t;
```

```

36     scanf("%d", &t);
37     for (; t > 0; t--) {
38         int l, r, k;
39         scanf("%d %d %d", &l, &r, &k);
40         int answer = prefix[r][k] - prefix[l-1][k];
41         printf("%d\n", answer);
42     }
43     return 0;
44 }

```

Ссылка на успешное отправление:

<https://codeforces.com/contest/932/submission/371785070>

Задание 107A:

В ориентированном графе, где у каждой вершины не более одного входящего и одного исходящего ребра, нужно найти все пути от источника (вершины с исходящим ребром, но без входящего) до стока (вершины с входящим ребром, но без исходящего) и для каждого такого пути определить минимальную пропускную способность (диаметр) на его рёбрах.

Ссылка на задание:

<https://codeforces.com/contest/107/problem/A>

В данном решении находятся все цепи от источников (вершин без входящих рёбер, но с исходящим) до стоков (вершин без исходящих) в графе, где каждая вершина имеет не более одного входящего и одного исходящего ребра. Для каждой цепи вычисляется минимальная пропускная способность среди её рёбер, после чего выводится количество таких цепей, а для каждой — номер источника, номер стока и найденный минимум.

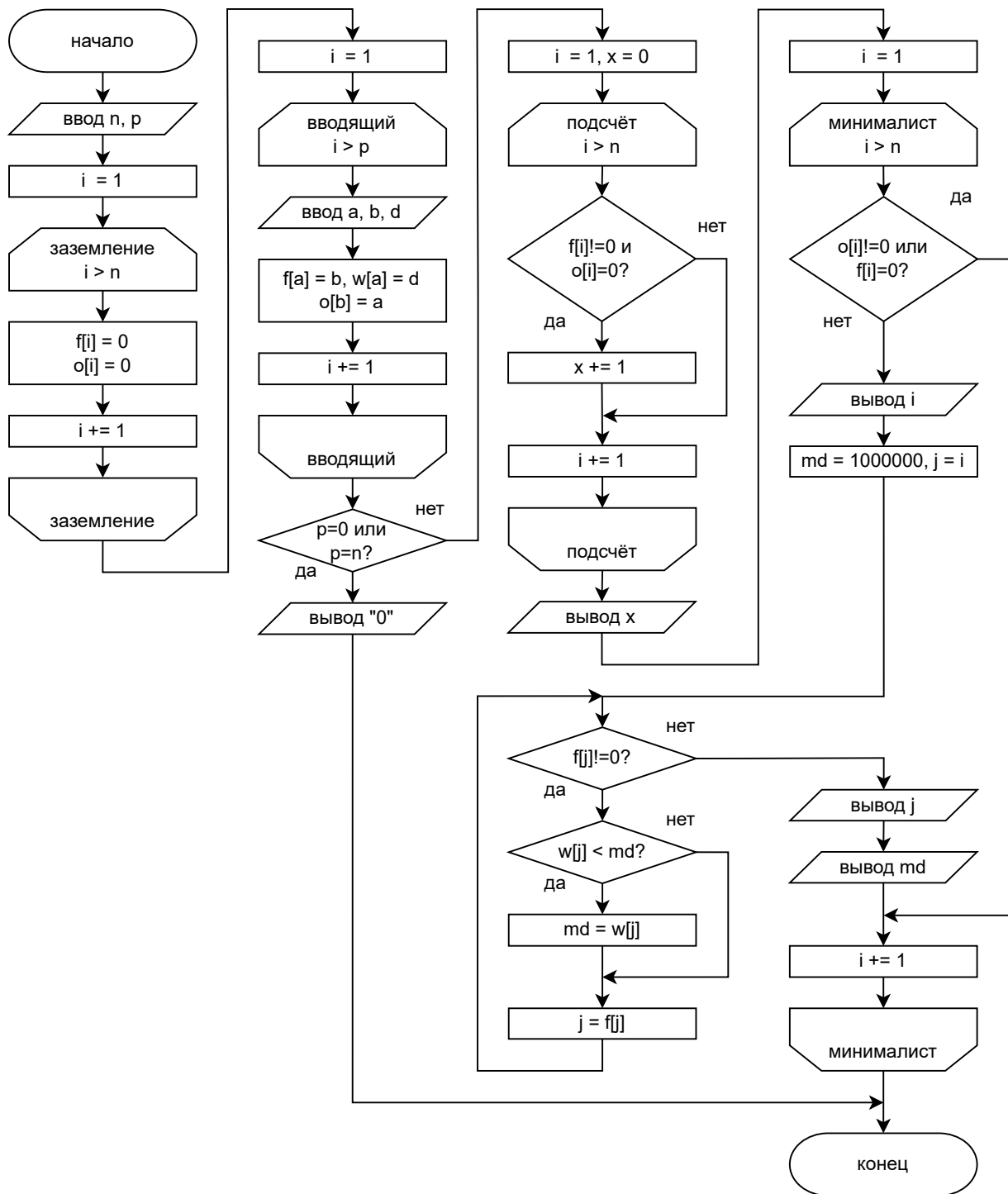


Рис. 5: Схема алгоритма задачи

Листинг 4: Исходный код программы (C)

```
1  #include <stdio.h>
2
3  int main() {
4      int n, p;
5      scanf("%d %d", &n, &p);
6
7      int o[n+1], f[n+1], w[n+1], x = 0;
8
9      for (int i = 1; i <= n; i++) {
10         f[i] = 0; o[i] = 0; }
11
12     for (int i = 1; i <= p; i++) {
13         int a, b, d;
14         scanf("%d %d %d", &a, &b, &d);
15         f[a] = b; w[a] = d; o[b] = a;
16     }
17
18     if (p == 0 || p == n) {
19         printf("0"); return 0; }
20
21     for (int i = 1; i <= n; i++)
22         if (f[i] != 0 && o[i] == 0) x++;
23     printf("%d\n", x);
24
25     for (int i = 1; i <= n; i++) {
26         if (o[i] != 0 || f[i] == 0) continue;
27         printf("%d ", i);
28         int md = 1000000, j = i;
29         while (f[j] != 0) {
30             if (w[j] < md) md = w[j];
31             j = f[j];
32         }
33         printf("%d ", j);
34         printf("%d\n", md);
35     }
```

```
36  
37     return 0;  
38 }
```

Выводы

В ходе выполнения лабораторной работы я научился применять несколько ключевых приёмов оптимизации, которые позволяют заменить прямое (часто переборное) решение на более эффективное.

Во-первых, использование жадного подхода вместо полного перебора вариантов. Например, в задаче с перелётами мы не перебираем все пары крупных городов, а ищем только ближайшие к пунктам А и В — это сокращает сложность с $O(k^2)$ до $O(k)$. А в задаче с кнопками движение «от цели к исходному числу» оказалось проще и быстрее, чем прямой BFS.

Во-вторых, предварительная обработка данных (preprocessing) и префиксные суммы. Для задачи с цифровыми корнями мы один раз вычислили ответ для всех чисел до миллиона и построили префиксные массивы по каждому корню. Благодаря этому каждый запрос обрабатывается за $O(1)$ вместо того, чтобы каждый раз заново считать корни на отрезке.

В-третьих, учёт структуры графа (не более одного входящего и одного исходящего ребра) позволил отказаться от сложных обходов вроде DFS — достаточно было пройти по цепочкам от источников к стокам, что дало линейную сложность.

Итог: главное, что я вынес — перед тем как писать «лобовое» решение, стоит проанализировать ограничения и найти закономерности. Жадность, предподсчёт, префиксные суммы и правильное использование условий задачи могут ускорить программу в сотни раз и уложиться в лимиты времени.